Problemas NP-Completos e Aproximações: Fundamentação, Algoritmos Exatos, Aproximados e Metaheurísticas em *Molecular Docking*

Igor, Maria Eduarda, Kairo Henrique, Heitor Henrique e Jade Souza
Departamento de Computação e Mecânica (DECOM-DV) – CEFET-MG, Divinópolis, MG
Estratégias Avançadas de Projeto e Análise de Algoritmos (Prof. Michel Pires da Silva)

***Resumo*—Este trabalho estuda os problemas NP-Completos e os métodos usados para lidar com o alto custo computacional de resolvê-los, tendo como base os Capítulos 34 e 35 do livro de Cormen et al. Na parte teórica, são apresentadas as classes P, NP, NP-Difícil e NP-Completo, a verificação em tempo polinomial por certificados, a discussão sobre P $\neq$ NP, as reduções polinomiais (CIRCUIT-SAT $\leq_P$ SAT $\leq_P$ 3-CNF-SAT $\leq_P$ CLIQUE $\leq_P$ VERTEX-COVER $\leq_P$ HAM-CYCLE $\leq_P$ TSP) e o cálculo da explosão combinatória. Na parte prática, são comparadas três estratégias implementadas em Python: algoritmos exatos (Força Bruta, *Branch-and-Bound* e Held-Karp), algoritmos de aproximação (`APPROX-VERTEX-COVER`, `APPROX-TSP-TOUR` e `GREEDY-SET-COVER`) e a metaheurística *Simulated Annealing* aplicada a um modelo simplificado de *Molecular Docking*.**

***Index Terms*—NP-Completude, Reduções Polinomiais, Explosão Combinatória, Algoritmos de Aproximação, *Simulated Annealing*.**

I. Introdução

Vários problemas clássicos de grafos possuem algoritmos exatos e rápidos. Para achar o menor caminho entre dois vértices com pesos não negativos, o algoritmo de Dijkstra resolve o problema em tempo $O(|E| + |V| \log |V|)$. Para conectar todos os vértices com o menor peso total de arestas, Prim e Kruskal encontram a Árvore Geradora Mínima em $O(|E| \log |V|)$ [1]. Nesses casos, o tempo de execução cresce de forma polinomial e permite processar grafos grandes em poucos segundos.

Por outro lado, existem problemas parecidos que não têm algoritmo eficiente conhecido. Se mudarmos o problema de caminhos mínimos para obrigar que a rota passe por todas as n cidades exatamente uma vez e volte ao início com o menor custo, temos o Problema do Caixeiro-Viajante (*Traveling Salesperson Problem* – TSP). Situação parecida acontece na Cobertura de Vértices (*Vertex Cover*), em que buscamos o menor conjunto de vértices que cubra todas as arestas do grafo, na Cobertura de Conjuntos (*Set Cover*) e no problema de encaixe molecular (*Molecular Docking*) [2], [3], onde se busca a posição e a rotação de uma molécula que minimizem a energia de ligação com uma proteína.

Na década de 1970, Cook [4], Karp [5] e Levin [6] provaram que esses problemas pertencem à classe NP-Completo (na forma de decisão) ou NP-Difícil (na forma de otimização). Todos eles são ligados por reduções em tempo polinomial: se alguém achar um algoritmo polinomial para um deles, todos os outros também terão solução polinomial.

Como não se conhece algoritmo polinomial para nenhum problema NP-Completo, achar a solução ótima exata exige testar um número de combinações que cresce de forma exponencial (2^n subconjuntos na Cobertura de Vértices) ou fatorial ($(n-1)!/2$ rotas no TSP simétrico). Com $n = 100$ cidades no TSP, existem cerca de $4{,}66 \times 10^{155}$ rotas, o que torna a busca exata inviável mesmo nos computadores mais rápidos.

Para contornar esse limite na prática, costumam ser usadas três abordagens:

1) *Algoritmos exatos com poda ou programação dinâmica:* métodos como *Branch-and-Bound* e Held-Karp [7] cortam buscas repetidas e acham o valor ótimo para entradas pequenas ($n \leq 20$), mas ainda sofrem com tempo exponencial e falta de memória RAM.
2) *Algoritmos de aproximação:* algoritmos polinomiais (Cap. 35 de Cormen et al. [1]) que entregam uma solução próxima da ótima com um limite matemático de erro $\rho(n)$.
3) *Metaheurísticas:* buscas probabilísticas como o *Simulated Annealing* [8], [9], usadas em problemas contínuos ou com muitos mínimos locais, onde não há garantia teórica de erro, mas busca-se uma boa solução dentro de um número fixo de avaliações.

O objetivo deste trabalho é estudar a base teórica da NP-Completude e comparar na prática o tempo de execução e a qualidade das soluções dessas três abordagens. Os objetivos específicos são: (i) apresentar as classes P, NP, NP-Difícil e NP-Completo, os verificadores polinomiais e a questão P $\neq$ NP; (ii) demonstrar as reduções da cadeia CIRCUIT-SAT $\leq_P$ SAT $\leq_P$ 3-CNF-SAT $\leq_P$ CLIQUE $\leq_P$ VERTEX-COVER $\leq_P$ HAM-CYCLE $\leq_P$ TSP e o crescimento de 2^n e $(n-1)!/2$; (iii) implementar soluções exatas e verificadores para TSP e Cobertura de Vértices; (iv) implementar os algoritmos de aproximação do Capítulo 35 e medir suas razões de aproximação; e (v) aplicar o *Simulated Annealing* ao estudo de caso inspirado em *Molecular Docking*.

O restante do texto está dividido da seguinte forma: a Seção II traz a fundamentação teórica; a Seção III apresenta os pseudocódigos e diagramas dos algoritmos; a Seção IV faz a análise assintótica de tempo e espaço e as provas de

aproximação; a Seção V mostra a implementação e os testes; a Seção VI resume as conclusões; e o Apêndice traz o guia rápido (*Cheat Sheet*).

II. Fundamentação Teórica

A. Problemas de Decisão e de Otimização

Na prática, problemas como o TSP ou a Cobertura de Vértices são de otimização, pois pedem o menor custo ou o menor conjunto possível. Porém, a teoria da NP-Completude do Capítulo 34 de Cormen et al. [1] trabalha com **problemas de decisão**, cuja resposta é apenas `SIM` (1) ou `NÃO` (0).

Todo problema de otimização pode ser transformado em um problema de decisão colocando um limite k sobre o valor buscado:

- `VERTEX-COVER`: dado um grafo não dirigido $G = (V, E)$ e um inteiro k, existe uma cobertura de vértices em G de tamanho no máximo k?
- `TSP`: dado um grafo completo $G = (V, E)$, os custos das arestas $c(i, j) \geq 0$ e um inteiro k, existe um ciclo hamiltoniano em G com custo total no máximo k?

Se fosse possível resolver a versão de otimização em tempo polinomial, bastaria achar o valor ótimo C^* e comparar se $C^* \leq k$ em $O(1)$ para responder ao problema de decisão. Logo, se o problema de decisão já for difícil, o de otimização é pelo menos tão difícil quanto ele. No livro, cada instância é codificada como uma cadeia binária $x \in \{0,1\}^*$, e o problema de decisão é visto como uma linguagem $L \subseteq \{0,1\}^*$ formada pelas entradas x que têm resposta `SIM` ($x \in L$).

B. Classes P e NP, Certificados e Verificadores

Definição 1 (Classe P [1])**.** A classe P é o conjunto das linguagens $L \subseteq \{0,1\}^*$ que podem ser decididas por um algoritmo determinístico em tempo polinomial $O(n^k)$, onde $n = |x|$ é o tamanho da entrada e k é uma constante.

Já a classe NP (*tempo polinomial não determinístico*) reúne os problemas cuja solução pode ser verificada em tempo polinomial.

Definição 2 (Verificador e Classe NP [1])**.** Um algoritmo de verificação $A(x, y)$ recebe a instância x e uma cadeia binária y chamada **certificado**. Uma linguagem L pertence a NP se existe um verificador de tempo polinomial A e uma constante c tais que:

$$L = \{x \in \{0,1\}^* : \exists y \text{ com } |y| = O(|x|^c) \text{ e } A(x, y) = 1\}. \tag{1}$$

Nos algoritmos implementados em `src/verification/verifier.py`, isso fica claro:

- Em `VERTEX-COVER`, o certificado é o subconjunto de vértices $V' \subseteq V$. O verificador confere se $|V'| \leq k$ e percorre as arestas $(u, v) \in E$ checando se $u \in V'$ ou $v \in V'$, gastando tempo $O(|V| + |E|)$.
- Em `TSP`, o certificado é a sequência de n vértices do ciclo. O verificador checa em $O(n)$ se todos os vértices aparecem uma única vez e se a soma dos pesos das arestas é menor ou igual a k.

Como qualquer problema em P já pode ser resolvido do zero em tempo polinomial sem precisar ler o certificado y, vale que P $\subseteq$ NP.

C. Reduções, NP-Difícil e NP-Completo

Definição 3 (Redução em Tempo Polinomial [1])**.** Uma linguagem L_1 é redutível em tempo polinomial a L_2 ($L_1 \leq_P L_2$) se existe uma função $f : \{0,1\}^* \to \{0,1\}^*$ calculável em tempo polinomial tal que, para todo $x \in \{0,1\}^*$:

$$x \in L_1 \iff f(x) \in L_2. \tag{2}$$

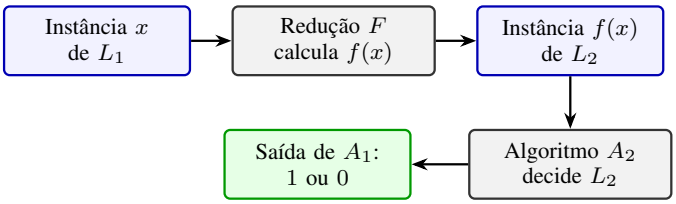


Figura 1. Ideia da redução polinomial $L_1 \leq_P L_2$ (Figura 34.5 de Cormen et al. [1]).

Lema 1 (Lema 34.3 de Cormen et al. [1])**.** *Se $L_1, L_2 \subseteq \{0,1\}^*$ são tais que $L_1 \leq_P L_2$, então $L_2 \in$ P $\implies$ $L_1 \in$ P.*

Demonstração. Sejam A_2 um algoritmo que decide L_2 em $O(|y|^k)$ e F o algoritmo de redução que gera $y = f(x)$ em $O(|x|^c)$ (Figura 1). Como F dá no máximo $O(|x|^c)$ passos, o tamanho de y é $|y| = O(|x|^c)$. Assim, $A_2(f(x))$ roda em tempo $O((|x|^c)^k) = O(|x|^{ck})$, e o algoritmo composto A_1 decide L_1 em tempo polinomial. □

Definição 4 (NP-Completo e NP-Difícil [1])**.** Uma linguagem $L \subseteq \{0,1\}^*$ é **NP-Completa** (NPC) se: (1) $L \in$ NP; e (2) $L' \leq_P L$ para todo $L' \in$ NP. Se L cumpre a condição 2 mas não precisa estar em NP, dizemos que L é **NP-Difícil**.

Teorema 1 (Teorema 34.4 de Cormen et al. [1])**.** *Se algum problema NP-Completo é resolvível em tempo polinomial, então P = NP. De modo equivalente, se algum problema em NP não está em P, então nenhum problema NP-Completo pode ser resolvido em tempo polinomial.*

Demonstração. Suponha que exista $L \in$ NPC tal que $L \in$ P. Para qualquer $L' \in$ NP, temos $L' \leq_P L$ pela propriedade 2 de NPC. Pelo Lema 1, $L \in$ P implica $L' \in$ P. Logo, NP $\subseteq$ P, e como P $\subseteq$ NP, segue que P = NP. A segunda parte do enunciado é a contrapositiva da primeira. □

D. A Questão P ≠ NP

Saber se P = NP ou P ≠ NP é um problema em aberto desde 1971 e um dos sete Problemas do Milênio do Instituto Clay. A dúvida central é se todo problema cuja resposta pode ser conferida rapidamente também pode ser resolvido do zero em tempo polinomial (Figura 2).

A maioria dos pesquisadores acredita que P ≠ NP porque milhares de problemas práticos já foram provados NP-Completos em áreas bem diferentes e, em mais de 50 anos de

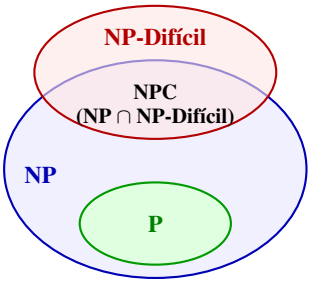


Figura 2. Relação entre P, NP, NPC e NP-Difícil supondo P $\neq$ NP [1].

estudo, nenhum algoritmo exato polinomial foi encontrado para nenhum deles [1]. Além disso, se P = NP com um algoritmo prático, haveria duas consequências diretas:

1) *Criptografia:* algoritmos como RSA e chaves baseadas em logaritmo discreto dependem do fato de que conferir uma chave ou multiplicar primos é rápido (P), enquanto fatorar ou inverter a operação sem a chave é custoso (NP). Se P = NP, essas chaves poderiam ser quebradas em tempo polinomial.
2) *Otimização:* problemas difíceis de logística, escala industrial e acoplamento molecular passariam a ter solução ótima exata em tempo polinomial.

E. Reduções Polinomiais e Demonstrações

Para provar que um novo problema L é NP-Completo sem reduzir toda a classe NP até ele, usa-se o Lema 34.8:

Lema 2 (Lema 34.8 de Cormen et al. [1])**.** *Se L' é NP-Completo e $L' \leq_P L$, então L é NP-Difícil. Se, além disso, $L \in$ NP, então $L \in$ NPC.*

Com isso, o roteiro de prova no Capítulo 34 de Cormen et al. [1] segue cinco passos: (1) provar que $L \in$ NP; (2) escolher um $L' \in$ NPC conhecido; (3) descrever a transformação $f(x)$; (4) provar que $x \in L' \iff f(x) \in L$; e (5) mostrar que f roda em tempo polinomial. A Figura 3 mostra a sequência de reduções usada até chegar em `VERTEX-COVER` e `TSP`.

1) De CIRCUIT-SAT para SAT e 3-CNF-SAT: O primeiro problema NP-Completo da cadeia é a satisfazibilidade de circuitos booleanos (`CIRCUIT-SAT`, Teorema 34.7 de Cormen et al. [1]). Para mostrar que toda linguagem $L \in$ NP se reduz a `CIRCUIT-SAT` (Lema 34.6), considera-se o verificador $A(x, y)$ que roda em tempo $T(n) = O(n^k)$. Em cada etapa da execução, o estado da memória, do contador de programa e dos registradores forma uma configuração c_i. Como a passagem de c_i para c_{i+1} é feita pelo hardware usando portas `AND`, `OR` e `NOT` (circuito M), basta ligar $T(n)$ cópias de M em sequência, fixar a entrada x e deixar os bits do certificado y como entradas livres. O circuito resultante $C = f(x)$ tem tamanho polinomial e produz saída 1 se e somente se existir um certificado y tal que $A(x, y) = 1$.

Para reduzir CIRCUIT-SAT $\leq_P$ SAT (Teorema 34.9), não podemos simplesmente substituir as entradas das portas de trás para frente, pois fios com leque de saída (*fan-out*) ≥ 2 duplicariam subfórmulas e fariam a fórmula crescer de forma exponencial. Em vez disso, cria-se uma variável x_i para cada fio e escreve-se uma cláusula com o operador $\leftrightarrow$ para cada porta (por exemplo, $x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)$ para uma porta `AND` na Figura 34.10 do livro). A fórmula final é o `AND` do fio de saída com as cláusulas de todas as portas.

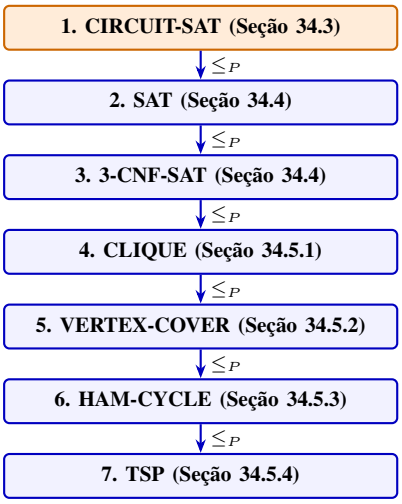


Figura 3. Sequência de reduções polinomiais do Capítulo 34 de Cormen et al. [1].

Depois, reduz-se SAT $\leq_P$ 3-CNF-SAT (Teorema 34.10), onde cada cláusula é um `OR` de exatamente três literais distintos. A conversão é feita em três etapas: (i) monta-se a árvore binária da fórmula criando variáveis y_i nos nós internos para que cada cláusula tenha no máximo 3 variáveis; (ii) usa-se a tabela-verdade de cada cláusula (máximo de 8 linhas) e as Leis de De Morgan para passá-la para a Forma Normal Conjuntiva (CNF); e (iii) usam-se duas variáveis auxiliares p e q para esticar cláusulas de 1 ou 2 literais (por exemplo, $(l_1 \vee l_2)$ vira $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$).

2) Redução 3-CNF-SAT $\leq_P$ CLIQUE: Um clique em $G = (V, E)$ é um subconjunto $V' \subseteq V$ onde todo par de vértices é ligado por uma aresta. O problema `CLIQUE` pergunta se existe um clique de tamanho k em G.

Teorema 2 (Teorema 34.11 de Cormen et al. [1])**.** *O problema CLIQUE é NP-Completo.*

Demonstração. Primeiro, CLIQUE $\in$ NP: dado o conjunto V' como certificado, confere-se se $|V'| = k$ e se toda dupla $u, v \in V'$ tem aresta em E em tempo $O(|V|^2)$.

Para reduzir 3-CNF-SAT $\leq_P$ CLIQUE, seja $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ uma fórmula em 3-CNF em que cada cláusula é $C_r = (l_1^r \vee l_2^r \vee l_3^r)$. Construímos o grafo $G = (V, E)$ assim:

- Para cada cláusula C_r, colocamos uma tripla de vértices $V_r = \{v_1^r, v_2^r, v_3^r\}$ em V ($3k$ vértices ao todo).
- Colocamos uma aresta entre v_i^r e v_j^s se: (a) eles estão em triplas diferentes ($r \neq s$); e (b) seus literais são coerentes, isto é, l_i^r não é a negação de l_j^s ($l_i^r \neq \neg l_j^s$).

Essa construção testa os pares entre os $3k$ vértices e roda em tempo polinomial $O(k^2)$. Resta provar a equivalência:

- *Ida ($\Longrightarrow$):* Se ϕ tem uma atribuição que a satisfaz, cada cláusula C_r tem pelo menos um literal verdadeiro $l_i^r = 1$. Escolhendo um vértice verdadeiro v_i^r de cada tripla, formamos V' com k vértices. Como dois vértices de V' têm literais iguais a 1 na mesma atribuição, um não pode ser a negação do outro. Logo, todos os pares em V' têm aresta em E, formando um clique de tamanho k.
- *Volta ($\Longleftarrow$):* Se G tem um clique V' de tamanho k, como não há arestas dentro da mesma tripla V_r, V' tem exatamente um vértice de cada uma das k triplas. Como os vértices de V' são todos ligados entre si, nenhum literal de V' é a negação de outro, permitindo atribuir 1 a todos os literais de V' sem conflito e satisfazer ϕ.

□

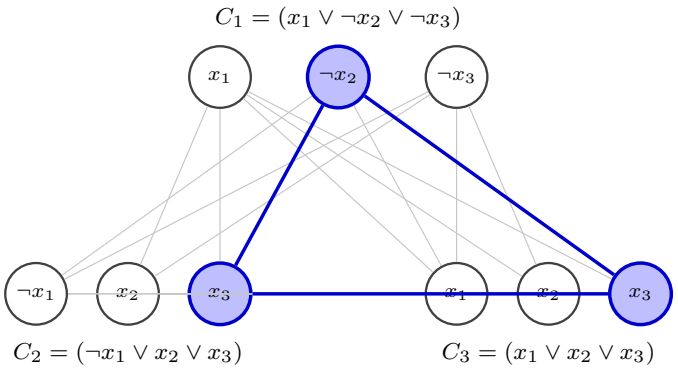


Figura 4. Redução 3-CNF-SAT $\leq_P$ CLIQUE da Figura 34.14 de Cormen et al. [1]. Para $x_2 = 0$ e $x_3 = 1$, os vértices $\{\neg x_2, x_3, x_3\}$ formam um clique de tamanho 3.

3) Redução CLIQUE $\leq_P$ VERTEX-COVER e TSP: Dado $G = (V, E)$, o grafo complemento é $\overline{G} = (V, \overline{E})$, com $\overline{E} = \{(u, v) : u \neq v \text{ e } (u, v) \notin E\}$ (Figura 5).

Teorema 3 (Teorema 34.12 de Cormen et al. [1])**.** *O problema VERTEX-COVER é NP-Completo.*

Demonstração. Já vimos que VERTEX-COVER $\in$ NP. Dada uma instância (G, k) de `CLIQUE`, calcula-se o complemento $\overline{G}$ em $O(|V|^2)$ e gera-se a instância $(\overline{G}, |V| - k)$ de `VERTEX-COVER`:

- *Ida ($\Longrightarrow$):* Se V' é um clique de tamanho k em G, então para toda aresta $(u, v) \in \overline{E}$ vale $(u, v) \notin E$. Logo, u e v não podem estar ambos em V', ou seja, pelo menos um deles pertence a $V \setminus V'$. Assim, $V \setminus V'$ cobre todas as arestas de $\overline{E}$ e tem tamanho $|V| - k$.
- *Volta ($\Longleftarrow$):* Se V' é uma cobertura de tamanho $|V| - k$ em $\overline{G}$, então para toda dupla $u, v \in V \setminus V'$ a aresta (u, v) não pode estar em $\overline{E}$ (pois não teria ponta em V'). Logo, $(u, v) \in E$ e $V \setminus V'$ é um clique de tamanho k em G.

□

Em seguida, o Teorema 34.13 reduz VERTEX-COVER $\leq_P$ HAM-CYCLE trocando cada aresta $(u, v) \in E$ por um *widget*

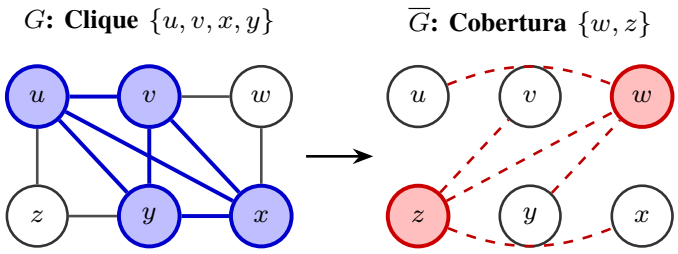


Figura 5. Redução CLIQUE $\leq_P$ VERTEX-COVER (Figura 34.15 de Cormen et al. [1]).

W_{uv} de 12 vértices ligado a k vértices seletores $s_1, \ldots, s_k$. Por fim, o Teorema 34.14 prova que HAM-CYCLE $\leq_P$ TSP: dado $G = (V, E)$, monta-se o grafo completo G' com custo $c(i, j) = 0$ se $(i, j) \in E$ e $c(i, j) = 1$ se $(i, j) \notin E$, e limite $k = 0$. Assim, G tem ciclo hamiltoniano se e somente se G' tem percurso de custo 0.

F. Explosão Combinatória

A dificuldade prática dos algoritmos de força bruta vem da quantidade de combinações possíveis:

1) *Cobertura de Vértices (2^n):* Como cada um dos $n = |V|$ vértices pode entrar ou não no subconjunto, o total de subconjuntos é $\sum_{k=0}^{n} \binom{n}{k} = 2^n$, levando a $O(2^n \cdot |E|)$.
2) *Caixeiro-Viajante ($(n-1)!/2$):* Fixando a primeira cidade entre as $n!$ permutações ($n!/n = (n-1)!$) e dividindo por 2 porque o sentido horário e anti-horário têm o mesmo custo em grafos simétricos, existem $\frac{(n-1)!}{2}$ rotas.

Aumentar a velocidade do processador ajuda pouco quando o algoritmo é exponencial. Se um algoritmo de tempo 2^n passa a rodar em um computador 1.000 vezes mais rápido, o novo tamanho resolvido no mesmo tempo é $2^{n'} = 1000 \cdot 2^n \implies n' = n + \log_2(1000) \approx n + 10$. Ou seja, ganham-se apenas 10 vértices. A Tabela I mostra o tempo estimado em um processador de 10^9 operações por segundo. Quando n cresce, usam-se algoritmos de aproximação com razão $\max(C/C^*, C^*/C) \leq \rho(n)$ (Cap. 35 [1]) ou metaheurísticas como o *Simulated Annealing*.

Tabela I
TEMPO ESTIMADO PARA 10^9 OP/S EM DIFERENTES COMPLEXIDADES.

n	Polinomial (n^2)	Exponencial (2^n)	Held-Karp ($n^2 2^n$)	Fatorial ($(n-1)!/2$)
5	25 ns	32 ns	0,8 μs	12 ns
10	100 ns	1,0 μs	102,4 μs	181,4 μs
15	225 ns	32,8 μs	7,4 ms	43,6 s
20	400 ns	1,0 ms	419,4 ms	1,9 anos
25	625 ns	33,6 ms	21,0 s	$9{,}8 \times 10^6$ anos
50	2,5 μs	13,0 dias	89,3 anos	$9{,}6 \times 10^{45}$ anos
100	10,0 μs	$4{,}0 \times 10^{13}$ anos	$4{,}0 \times 10^{17}$ anos	$1{,}5 \times 10^{139}$ anos

III. O Paradigma Algorítmico (Engenharia da Solução)

A Seção II mostrou por que não se espera um algoritmo exato e polinomial para problemas NP-Completos. Esta seção apresenta o projeto dos algoritmos, isto é, como cada um

resolve o problema. Na prática, há duas estratégias. A primeira busca o ótimo de forma sistemática, com poda ou programação dinâmica, o que só é viável para entradas pequenas. A segunda aceita uma solução de qualidade um pouco inferior, calculada em tempo polinomial, com um limite provado para o erro. Esses algoritmos foram implementados no repositório da equipe (Seção V).

A. Algoritmos Exatos

1) Branch-and-Bound para a Cobertura de Vértices: O método parte de uma propriedade da cobertura de vértices. Para qualquer aresta (u, v) ainda descoberta, a cobertura precisa conter u ou v. Assim, a busca gera dois ramos, um com u e outro com v. Para não explorar ramos que não podem melhorar a resposta, o algoritmo mantém a melhor cobertura encontrada e descarta (poda) todo ramo que atinge o mesmo número de vértices dela. Qualquer cobertura válida serve como limitante inicial. No pseudocódigo usamos a resposta do `APPROX-VERTEX-COVER` (Algoritmo 3), que é rápida. O código da equipe usa outro método guloso, que escolhe sempre o vértice de maior grau, e o resto da busca é igual.

Algoritmo 1: `BB-VERTEX-COVER`(G)

Entrada: Grafo não dirigido $G = (V, E)$.
Saída: Cobertura de vértices mínima C^*.

$melhor \leftarrow$ `APPROX-VERTEX-COVER`(G)
`BUSCA`$(\emptyset)$
retorna $melhor$

Procedimento `BUSCA`(C)
se $|C| \geq |melhor|$ **então**
  retorna `// poda`
procure $(u, v) \in E$ com $u \notin C$ e $v \notin C$
se *não existe tal aresta* **então**
  $melhor \leftarrow C$
  retorna
`BUSCA`$(C \cup \{u\})$ `// ramo 1`
`BUSCA`$(C \cup \{v\})$ `// ramo 2`

A Figura 6 mostra a árvore de busca no grafo com arestas $(1, 2)$, $(1, 3)$, $(2, 3)$ e $(3, 4)$. O limitante inicial do `APPROX-VERTEX-COVER` é $\{1, 2, 3, 4\}$, com 4 vértices. A busca encontra $\{1, 2, 3\}$, depois $\{1, 3\}$, e a partir daí poda todos os ramos com 2 vértices ou mais.

2) Força Bruta e Held-Karp para o TSP: A força bruta fixa a cidade inicial 1, testa todas as ordens das outras $n - 1$ cidades e mantém a rota de menor custo. O algoritmo de Held-Karp [7] usa programação dinâmica para não recalcular subproblemas que se repetem. Ele guarda em $D[S, k]$ o custo do menor caminho que sai da cidade 1, passa exatamente pelas cidades do conjunto S e termina na cidade $k \in S$.

O código da equipe preenche a mesma tabela D na mesma ordem, do menor conjunto para o maior. Cada conjunto S fica guardado como uma *bitmask*, um número inteiro em que o bit i vale 1 quando a cidade i está em S. O custo é o mesmo do

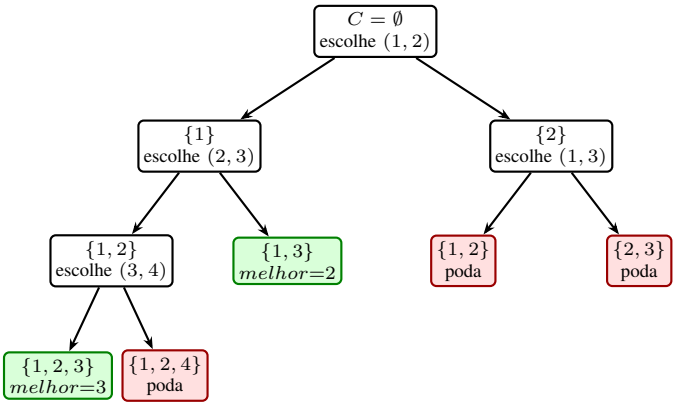


Figura 6. Árvore do `BB-VERTEX-COVER`. Verde é uma nova melhor solução e vermelho é um ramo podado. O resultado é $C^* = \{1, 3\}$.

Algoritmo 2: `HELD-KARP`(n, c) [7]

Entrada: n cidades e custos $c(i, j)$.
Saída: Custo do tour ótimo.

para $k \leftarrow 2$ **até** n **faça**
  $D[\{k\}, k] \leftarrow c(1, k)$
para $s \leftarrow 2$ **até** $n - 1$ **faça**
  para cada $S \subseteq \{2, \ldots, n\}$ *com* $|S| = s$ **faça**
    para cada $k \in S$ **faça**
      $D[S, k] \leftarrow \min_{m \in S - \{k\}} D[S - \{k\}, m] + c(m, k)$
retorna $\min_k D[\{2, \ldots, n\}, k] + c(k, 1)$

Algoritmo 2. A solução é construída de forma incremental. O menor custo para chegar em k após visitar S depende apenas dos menores custos para chegar em cada cidade m anterior a k, valores que já estão na tabela.

B. Algoritmos de Aproximação

Os três algoritmos desta parte vêm do Capítulo 35 de Cormen et al. [1]. Todos tomam decisões locais e nunca desfazem uma escolha. A garantia de qualidade de cada um é provada na Seção IV.

1) `APPROX-VERTEX-COVER`: O algoritmo escolhe uma aresta ainda descoberta e insere *as duas extremidades* na cobertura. Embora uma extremidade fosse suficiente para cobrir essa aresta, é essa escolha que garante o fator 2.

A Figura 7 mostra o laço como fluxograma e a Figura 8 mostra a execução no grafo da Figura 35.1 de Cormen et al. [1].

2) `APPROX-TSP-TOUR`: Este algoritmo só tem garantia quando os custos obedecem à *desigualdade triangular*, $c(u, w) \leq c(u, v) + c(v, w)$. Ou seja, o caminho direto de u até w nunca custa mais do que um caminho com parada intermediária em v, como ocorre com distâncias em linha reta. O algoritmo constrói a Árvore Geradora Mínima (MST),

Algoritmo 3: APPROX-VERTEX-COVER(G) [1]

Entrada: Grafo não dirigido $G = (V, E)$.
Saída: Cobertura de vértices $C \subseteq V$ com $|C| \leq 2|C^*|$.

$C \leftarrow \emptyset$
$E' \leftarrow E[G]$
enquanto $E' \neq \emptyset$ **faça**
  Escolha uma aresta qualquer $(u, v) \in E'$
  $C \leftarrow C \cup \{u, v\}$
  Remova de E' todas as arestas incidentes a u ou v
retorna C

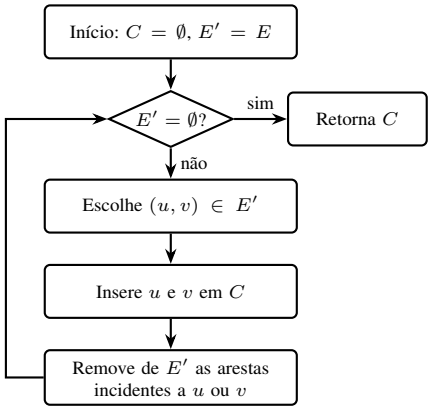
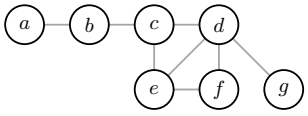


Figura 7. Fluxograma do APPROX-VERTEX-COVER.

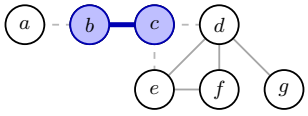
percorre essa árvore em pré-ordem e visita as cidades na ordem da primeira visita, com retorno à cidade inicial ao final.

Observe o trecho de f para e na Figura 9(b). Na árvore, o caminho de f até e passa por c e b. O tour segue diretamente de f para e, e a desigualdade triangular garante que esse atalho não aumenta o custo. Esse argumento é a base da prova na Seção IV.

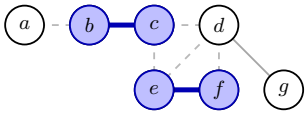
3) GREEDY-SET-COVER: Na Cobertura de Conjuntos, a entrada é um conjunto X e uma família $\mathcal{F}$ de subconjuntos de X. O algoritmo guloso escolhe, a cada rodada, o subconjunto



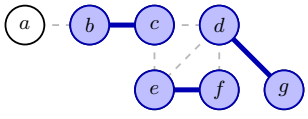
(a) Grafo inicial, $C = \emptyset$.



(b) Escolhe (b, c). Cobre ab, cd, ce.



(c) Escolhe (e, f). Cobre de, df.



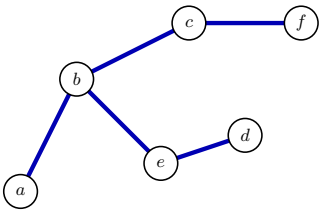
(d) Escolhe (d, g). Fim, $|C| = 6$.

Figura 8. Execução do APPROX-VERTEX-COVER. Vértices azuis estão em C, arestas em destaque são as escolhidas e arestas tracejadas já foram cobertas. A cobertura ótima é $\{b, d, e\}$, com 3 vértices, então o algoritmo usou exatamente o dobro.

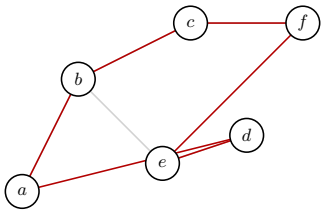
Algoritmo 4: APPROX-TSP-TOUR(G, c) [1]

Entrada: Grafo completo $G = (V, E)$ com custos c que obedecem à desigualdade triangular.
Saída: Ciclo hamiltoniano H com $c(H) \leq 2\, c(H^*)$.

Escolha um vértice $r \in V$ como raiz
$T \leftarrow$ MST-PRIM(G, c, r)
$L \leftarrow$ vértices de T em pré-ordem a partir de r
$H \leftarrow L$ seguida de r
retorna H



(a) MST com raiz a



(b) Tour a, b, c, f, e, d, a

Figura 9. Execução do APPROX-TSP-TOUR com distância euclidiana. A pré-ordem da MST é a, b, c, f, e, d. O tour custa cerca de 15,7 e o ótimo (a, b, c, f, d, e, a) custa cerca de 12,8, uma razão de 1,22, abaixo do limite 2.

que cobre o *maior número de elementos ainda descobertos*.

Algoritmo 5: GREEDY-SET-COVER$(X, \mathcal{F})$ [1]

Entrada: Conjunto X e família $\mathcal{F}$ de subconjuntos de X.
Saída: Família $\mathcal{C} \subseteq \mathcal{F}$ que cobre X.

$U \leftarrow X$ `// elementos que faltam`
$\mathcal{C} \leftarrow \emptyset$
enquanto $U \neq \emptyset$ **faça**
  Escolha $S \in \mathcal{F}$ que maximiza $|S \cap U|$
  $U \leftarrow U - S$
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{S\}$
retorna $\mathcal{C}$

A Tabela II mostra um caso em que o guloso *não* encontra o ótimo. Temos $X = \{1, \ldots, 6\}$, $A = \{1, 2, 3\}$, $B = \{4, 5, 6\}$ e $D = \{1, 2, 4, 5\}$. O ótimo é $\{A, B\}$, mas o guloso começa por D, que cobre quatro elementos, e depois precisa de mais dois conjuntos. Como a decisão considera apenas a rodada atual, o algoritmo não percebe que a escolha de D impede uma solução com dois conjuntos.

Tabela II
EXECUÇÃO DO GREEDY-SET-COVER. CADA COLUNA MOSTRA QUANTOS ELEMENTOS NOVOS O CONJUNTO COBRIRIA NA RODADA.

Rodada	U antes	$\lvert A \cap U \rvert$	$\lvert B \cap U \rvert$	$\lvert D \cap U \rvert$	Escolhe
1	$\{1, \ldots, 6\}$	3	3	**4**	D
2	$\{3, 6\}$	**1**	1	0	A
3	$\{6\}$	0	**1**	0	B

IV. Análise Assintótica Rigorosa (Custos)

Nesta seção medimos o custo de tempo e de memória de cada algoritmo da Seção III e depois provamos os fatores de aproximação. Usamos $|V|$ e $|E|$ para grafos e n para o número de cidades do TSP. Quando falamos de memória, contamos só o espaço *extra*, além da própria entrada.

A. As Notações O, Ω e Θ

Como o tempo em segundos depende da máquina, a análise considera a forma como o custo *cresce* em função do tamanho da entrada [1].

Definição 5 (Notações assintóticas [1])**.** Para funções $f(n)$ e $g(n)$ positivas: $f(n) = O(g(n))$ se existem $c > 0$ e n_0 com $f(n) \le c\,g(n)$ para todo $n \ge n_0$; $f(n) = \Omega(g(n))$ se existem $c > 0$ e n_0 com $f(n) \ge c\,g(n)$ para todo $n \ge n_0$; e $f(n) = \Theta(g(n))$ se valem as duas ao mesmo tempo.

De forma intuitiva, O indica um limite superior ("no máximo"), Ω um limite inferior ("no mínimo") e Θ um limite justo ("exatamente nessa ordem"). Por exemplo, $f(n) = 3n^2 + 5n$ é $\Theta(n^2)$. Com $c = 4$ e $n_0 = 5$ temos $3n^2 + 5n \le 4n^2$, e com $c = 3$ temos $3n^2 + 5n \ge 3n^2$ para todo $n \ge 1$.

B. Custo dos Algoritmos Exatos

Cobertura de Vértices por força bruta. Como visto na Seção II, existem $2^{|V|}$ subconjuntos, e o verificador gasta $O(|V|+|E|)$ em cada um. O tempo é $O(2^{|V|} \cdot |E|)$ para grafos sem vértices isolados, e a memória extra é $O(|V|)$ para marcar o subconjunto atual.

`BB-VERTEX-COVER`. Seja k o tamanho do limitante inicial. Cada chamada de `BUSCA` acrescenta um vértice a C, e a poda interrompe o ramo quando $|C|$ chega a k. Então todo nó que não é podado tem $|C| < k$, ou seja, fica a uma profundidade menor que k. Como cada nó gera no máximo dois filhos, existem menos de 2^k desses nós. Em cada um deles a procura da aresta descoberta percorre E, o que custa $O(|E|)$, e cada nó podado custa só $O(1)$. Assim,

$$T_{BB}(G) = O(2^k \cdot |E|). \tag{3}$$

Com o limitante do `APPROX-VERTEX-COVER` temos $k \le 2|C^*|$ (Teorema 4), então o custo depende do tamanho da resposta e não do número de vértices. O guloso por grau do código não tem essa garantia de fator 2, então lá o valor de k depende da instância. A memória extra é $O(|V|)$, pois a pilha da recursão tem altura menor que $k \le |V|$ e C é um vetor de marcação. No pior caso, porém, k cresce junto com $|V|$ e o tempo volta a ser exponencial.

TSP por força bruta. Com a cidade 1 fixa, existem $(n-1)!$ ordens, e somar o custo de cada rota leva $\Theta(n)$. O tempo é $\Theta(n\cdot(n-1)!) = \Theta(n!)$. O código testa todas as $(n-1)!$ ordens. Se ele descartasse as rotas no sentido contrário, sobrariam $(n-1)!/2$, mas a ordem de crescimento seria a mesma. A memória extra é $O(n)$ para a permutação atual.

`HELD-KARP`. A tabela D tem um valor para cada par (S, k) com $S \subseteq \{2, \dots, n\}$ e $k \in S$, ou seja, no máximo $(n-1)\,2^{n-1}$ valores. Cada valor testa no máximo n vizinhos m. Logo,

$$T_{HK}(n) = \Theta(n^2\,2^n), \qquad M_{HK}(n) = \Theta(n\,2^n). \tag{4}$$

O tempo é muito menor que $n!$, mas o algoritmo troca tempo por memória, que agora também cresce de forma exponencial. Isso explica o estouro de memória RAM nos experimentos. Com $n = 22$ a tabela tem mais de 2×10^7 valores e com $n = 25$ passa de 2×10^8, e em Python cada valor ocupa dezenas de bytes. A Tabela I compara essas funções em segundos.

C. Custo dos Algoritmos de Aproximação

`APPROX-VERTEX-COVER`. Com listas de adjacência, cada aresta sai de E' uma única vez e cada vértice entra em C no máximo uma vez. O tempo é $O(|V| + |E|)$ [1] e a memória extra é $O(|V|)$ para marcar os vértices de C, pois uma aresta está em E' se e somente se nenhuma das suas extremidades está marcada.

`APPROX-TSP-TOUR`. O grafo é completo, então $|E| = \Theta(n^2)$. O passo mais caro é o `MST-PRIM`. Na versão com vetor simples ele faz n rodadas, e cada rodada procura o menor valor entre n cidades, o que dá $\Theta(n^2)$. A pré-ordem é uma busca em profundidade numa árvore de $n-1$ arestas e custa $\Theta(n)$. O total é $\Theta(n^2)$ [1]. Se as distâncias forem calculadas a partir das coordenadas, a memória extra é $O(n)$ para os vetores de chave e de pai do Prim e para a lista L.

`GREEDY-SET-COVER`. O laço roda no máximo $\min(|X|, |\mathcal{F}|)$ vezes, pois cada rodada cobre pelo menos um elemento novo e usa um conjunto que ainda não estava em $\mathcal{C}$. Cada rodada calcula $|S \cap U|$ para todos os conjuntos, com custo $O(|X|\,|\mathcal{F}|)$. Logo [1],

$$T_{GSC} = O\big(|X|\,|\mathcal{F}|\min(|X|, |\mathcal{F}|)\big). \tag{5}$$

A memória extra é $O(|X|)$ para o conjunto U e para $\mathcal{C}$, que tem no máximo $|X|$ conjuntos.

Os três custos são polinomiais. Essa é a principal vantagem das aproximações sobre os métodos exatos, e a Tabela do Apêndice A resume todos os custos lado a lado.

D. Provas dos Fatores de Aproximação

Um algoritmo tem fator de aproximação $\rho(n)$ quando o custo C da sua resposta e o ótimo C^* obedecem a $\max(C/C^*, C^*/C) \le \rho(n)$ para toda entrada de tamanho n [1]. Em problemas de minimização isso é o mesmo que $C \le \rho(n) \cdot C^*$. Um fator 2 significa que a resposta custa no máximo o dobro da melhor possível.

Teorema 4 (Teorema 35.1 de Cormen et al. [1])**.** *O* `APPROX-VERTEX-COVER` *é uma 2-aproximação de tempo polinomial.*

Demonstração. O tempo $O(|V| + |E|)$ já foi mostrado. A resposta é uma cobertura válida, pois o laço só termina quando $E' = \emptyset$, e uma aresta só sai de E' quando uma de suas extremidades entra em C.

Seja A o conjunto das arestas escolhidas na linha 4 do Algoritmo 3. Quando (u, v) é escolhida, todas as arestas incidentes a u ou v saem de E'. Assim, duas arestas de A nunca têm extremidade em comum, ou seja, A é um *emparelhamento*. Ele é *maximal* porque nenhuma outra aresta de E pode entrar em A sem compartilhar uma extremidade. Na Figura 8, por exemplo, $A = \{(b, c), (e, f), (d, g)\}$ e as seis extremidades são todas diferentes.

Toda cobertura ótima C^* precisa ter pelo menos uma extremidade de cada aresta de A. Como essas arestas não compartilham extremidades, um mesmo vértice de C^* não cobre duas delas. Logo $|C^*| \geq |A|$. Cada aresta de A coloca dois vértices novos em C, então $|C| = 2|A|$. Com as duas desigualdades, $|C| = 2|A| \leq 2|C^*|$. □

Um aspecto importante dessa prova é que ela não depende de conhecer C^*. Basta o limite inferior $|C^*| \geq |A|$.

Teorema 5 (Teorema 35.2 de Cormen et al. [1])**.** *Com custos que obedecem à desigualdade triangular, o* `APPROX-TSP-TOUR` *é uma 2-aproximação de tempo polinomial.*

Demonstração. O tempo $\Theta(n^2)$ já foi mostrado. Sejam H^* um tour ótimo e T a MST.

Passo 1. Ao remover uma aresta de H^*, resta um caminho que liga todas as cidades, e esse caminho é uma árvore geradora. Como os custos não são negativos e T é a árvore geradora mais barata, $c(T) \leq c(H^*)$.

Passo 2. Seja W o passeio completo pela árvore, que percorre cada aresta uma vez na descida e outra vez na volta. Na Figura 9, $W = a, b, c, f, c, b, e, d, e, b, a$. Cada aresta de T aparece duas vezes, então $c(W) = 2\,c(T)$.

Passo 3. O tour H é o próprio W sem as visitas repetidas. Cada remoção de um vértice repetido transforma um trecho $u \to v \to w$ na aresta direta $u \to w$, e pela desigualdade triangular o custo não aumenta. Logo $c(H) \leq c(W)$.

Com os três passos, $c(H) \leq c(W) = 2\,c(T) \leq 2\,c(H^*)$. □

A hipótese da desigualdade triangular é essencial. Pelo Teorema 35.3 de Cormen et al. [1], se P $\neq$ NP, não existe aproximação de tempo polinomial com nenhum fator constante $\rho \geq 1$ para o TSP geral. A ideia da prova é que um algoritmo assim conseguiria decidir `HAM-CYCLE`, que é NP-Completo (Seção II). Esse é um exemplo direto de como a questão P $\neq$ NP limita até as aproximações.

Para a Cobertura de Conjuntos o fator não é constante. Ele usa o número harmônico $H(d) = \sum_{i=1}^{d} 1/i \leq \ln d + 1$.

Teorema 6 (Teorema 35.4 e Corolário 35.5 de Cormen et al. [1])**.** *O* `GREEDY-SET-COVER` *é uma $H(d)$-aproximação de tempo polinomial, em que $d = \max\{|S| : S \in \mathcal{F}\}$. Em particular, o fator é no máximo* $\ln |X| + 1$.

Demonstração. A ideia é dividir o custo de cada conjunto escolhido entre os elementos que ele cobriu pela primeira vez. Se o guloso escolhe um conjunto que cobre t elementos novos, cada um deles recebe o preço $c_x = 1/t$. Como cada conjunto escolhido distribui exatamente 1, vale $|\mathcal{C}| = \sum_{x \in X} c_x$.

Agora fixe qualquer $S \in \mathcal{F}$ e considere seus elementos na ordem em que foram cobertos. Suponha que, num certo momento, ainda faltam u elementos de S. Nesse momento o próprio S cobriria u elementos novos, então o guloso escolheu um conjunto que cobre *pelo menos* u elementos novos. Assim o próximo elemento de S a ser coberto paga no máximo $1/u$. O primeiro paga no máximo $1/|S|$, o segundo no máximo $1/(|S| - 1)$, e assim até o último, que paga no máximo 1. A soma dos preços é

$$\sum_{x \in S} c_x \leq \frac{1}{|S|} + \frac{1}{|S| - 1} + \cdots + 1 = H(|S|) \leq H(d). \quad (6)$$

A cobertura ótima $\mathcal{C}^*$ contém cada elemento de X em pelo menos um de seus conjuntos. Portanto

$$|\mathcal{C}| = \sum_{x \in X} c_x \leq \sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x \leq |\mathcal{C}^*| \cdot H(d). \quad (7)$$

Como $d \leq |X|$, segue que $H(d) \leq \ln |X| + 1$. □

No exemplo da Tabela II, o maior conjunto tem $d = 4$, então a garantia é $H(4) \approx 2{,}08$. O guloso usou 3 conjuntos contra 2 do ótimo, uma razão de 1,5, dentro do limite. As razões medidas no repositório (Seção VI) também ficaram abaixo de todos os limites provados aqui.

Quando nenhuma garantia desse tipo é conhecida, como no *Molecular Docking*, que é um problema contínuo, a alternativa são as metaheurísticas da Seção V. Elas costumam encontrar boas soluções na prática, mas não oferecem prova de proximidade do ótimo.

V. Estudo de Caso e Implementação Prática

Esta seção aplica as ideias anteriores a um problema contínuo inspirado em *Molecular Docking*, a busca pela posição e pela orientação de uma molécula pequena, o ligante, em relação a um alvo [2]. O modelo é educacional e simplificado. Ele não calcula afinidade química real, mas preserva a dificuldade central do problema, que é um espaço de busca contínuo com muitos mínimos locais. A documentação completa do modelo está no repositório da equipe.

A. Modelo do Problema

Uma solução, chamada de *pose*, é o vetor $p = (x, y, \theta)$, em que (x, y) é a posição do ligante no plano e θ é o ângulo de rotação. A busca ocorre na região $-4 \leq x, y \leq 4$ e $-\pi \leq \theta < \pi$. O ligante tem quatro átomos fixos, representados pelos pontos (0; 0), (0,8; 0), (0; 0,8) e (0,8; 0,8). Para avaliar uma pose, cada átomo (a_i, b_i) é rotacionado por θ e transladado por (x, y):

$$\begin{aligned} x_i' &= x + a_i \cos\theta - b_i \sin\theta, \\ y_i' &= y + a_i \sin\theta + b_i \cos\theta. \end{aligned} \quad (8)$$

Cada instância tem K alvos, sorteados em $[-2, 2]^2$, e $2K$ obstáculos, sorteados em $[-4, 4]^2$, em que K é o parâmetro de complexidade. O sorteio usa uma semente explícita, então

a mesma combinação de K e semente gera sempre a mesma instância. A energia de uma pose soma três parcelas:

$$
\begin{aligned}
E(p) &= E_{\text{atr}}(p) + E_{\text{rep}}(p) + E_{\text{lim}}(p), \qquad (9)\\
E_{\text{atr}}(p) &= -\sum_i e^{-d_i^2/(2\sigma^2)},\\
E_{\text{rep}}(p) &= \lambda \sum_i \sum_j \max(0,\, r - d_{ij})^2,\\
E_{\text{lim}}(p) &= \textstyle\sum_i \big[\max(0,\, |x_i'| - \ell)^2 + \max(0,\, |y_i'| - \ell)^2\big].
\end{aligned}
$$

Em E_{atr}, d_i é a distância do átomo i até o alvo mais próximo, e cada átomo contribui com um valor entre -1 e 0, mais negativo quanto mais perto ele está de um alvo. Em E_{rep}, d_{ij} é a distância do átomo i ao obstáculo j, e a penalidade só aparece quando essa distância fica abaixo de r. A parcela E_{lim} penaliza átomos que saem do quadrado $[-\ell, \ell]^2$. O código usa $\sigma = 1{,}2$, $r = 0{,}9$, $\lambda = 8$ e $\ell = 5$. Quanto menor a energia, melhor a pose, e o objetivo é encontrar $p^* = \arg\min_p E(p)$.

B. Metaheurística: Simulated Annealing

Como o espaço de busca da Seção V-A é contínuo e tem vários mínimos locais, não existe uma aproximação com fator ρ provado, e a estratégia adotada é o *Simulated Annealing* [8].

O método se inspira no resfriamento lento de um metal. A cada iteração, o algoritmo gera uma pose vizinha da atual, com um pequeno deslocamento aleatório em x, y e θ. Se a energia diminui, a nova pose é aceita. Se a energia aumenta em $\Delta > 0$, a pose ainda pode ser aceita com probabilidade

$$P(\text{aceitar}) = e^{-\Delta/T}, \qquad (10)$$

que é o critério de Metropolis [9]. A temperatura T começa alta e diminui de forma geométrica ao longo da execução. Por exemplo, para uma piora $\Delta = 0{,}5$, a probabilidade de aceitação é $e^{-0{,}25} \approx 0{,}78$ com $T = 2$, mas cai para $e^{-50} \approx 2 \times 10^{-22}$ com $T = 0{,}01$. No início, portanto, o algoritmo aceita pioras com frequência e explora regiões distantes. No final, ele quase só aceita melhoras e refina a melhor região encontrada. É esse mecanismo que permite sair de mínimos locais.

As energias de p e de p^* ficam guardadas em variáveis, então cada iteração do Algoritmo 6 calcula E uma única vez, para a pose p'. No código da equipe, os valores usados são $T_0 = 2$ e $T_f = 0{,}01$. O deslocamento da vizinha segue uma distribuição normal com desvio padrão $0{,}6$ em x e em y e $0{,}25$ rad em θ. A posição é mantida dentro de $[-4, 4]$ e o ângulo volta ao intervalo $[-\pi, \pi)$.

Custo. A instância tem K alvos e $2K$ obstáculos, e o ligante tem 4 átomos, um número fixo. Para cada átomo, a função de energia calcula a distância até todos os alvos e até todos os obstáculos, então uma avaliação de E custa $\Theta(K)$. Seja $B = N + 1$ o orçamento de avaliações, isto é, o número total de avaliações, contada a pose inicial. As demais operações de cada iteração custam $O(1)$, logo

$$T_{SA} = \Theta(B \cdot K). \qquad (11)$$

A memória extra é $O(K)$, ocupada pela lista de distâncias até os alvos dentro do cálculo da energia. Com o registro apenas da menor distância, a memória extra cairia para $O(1)$, sem mudança no tempo.

Algoritmo 6: SIMULATED-ANNEALING(E, N, T_0, T_f) [8]

Entrada: Função de energia E, número de iterações $N \geq 2$ e temperaturas $T_0 \geq T_f > 0$.
Saída: Melhor pose encontrada p^*.

$p \leftarrow$ pose aleatória (x, y, θ)
$p^* \leftarrow p$
para $i \leftarrow 0$ **até** $N - 1$ **faça**
  $T \leftarrow T_0\,(T_f/T_0)^{\,i/(N-1)}$ // resfriamento geométrico
  $p' \leftarrow p$ com deslocamento aleatório em x, y e θ
  $\Delta \leftarrow E(p') - E(p)$
  se $\Delta < 0$ ***ou*** ALEATÓRIO$(0, 1) < e^{-\Delta/T}$ **então**
    $p \leftarrow p'$ // critério de Metropolis
  se $E(p) < E(p^*)$ **então**
    $p^* \leftarrow p$
retorna p^*

Ao contrário dos algoritmos da Seção IV, o *Simulated Annealing* não tem um fator de aproximação provado. A qualidade da resposta depende do orçamento B e das temperaturas T_0 e T_f, e por isso é avaliada de forma experimental, em comparação com uma busca aleatória que usa o mesmo número de avaliações.

C. Busca Aleatória como Referência

A referência para medir o ganho do *Simulated Annealing* é a busca aleatória (*Random Search*). A cada avaliação, ela sorteia uma pose em toda a região de busca, calcula a energia e guarda a melhor pose. Ela não tem pose atual, vizinhança, critério de aceitação nem temperatura. Com B avaliações, o custo também é $\Theta(B \cdot K)$, então os dois métodos podem ser comparados com o mesmo orçamento.

D. Metodologia Experimental

Os experimentos cruzam cinco níveis de complexidade, $K \in \{2, 4, 6, 8, 10\}$, com cinco orçamentos, $B \in \{10^2, 10^3, 10^4, 10^5, 10^6\}$. Cada combinação foi repetida 30 vezes, o que resulta em 750 pares de execuções. Em cada par, os dois métodos usam a mesma instância e o mesmo orçamento, e todas as sementes são explícitas, o que torna os resultados reproduzíveis. Para cada par, calcula-se a diferença

$$D = E_{\text{SA}} - E_{\text{RS}}, \qquad (12)$$

em que $D < 0$ indica vitória do *Simulated Annealing*, pois a energia é minimizada. Em cada combinação, a análise usa duas ferramentas. A primeira é um intervalo de confiança de 95% para a média de D, obtido por *bootstrap*, isto é, por reamostragem com reposição das 30 diferenças. A segunda é o teste exato dos sinais, que verifica se o número de vitórias de um método se afasta do esperado sob a hipótese de chances iguais.

Tabela III
VITÓRIAS DO *Simulated Annealing* SOBRE A BUSCA ALEATÓRIA EM 30 EXECUÇÕES PAREADAS. EM NEGRITO, O INTERVALO DE CONFIANÇA DE 95% DA MÉDIA DE D FICA ABAIXO DE ZERO E FAVORECE O *Simulated Annealing*. COM †, ELE FICA ACIMA DE ZERO E FAVORECE A BUSCA ALEATÓRIA. NOS DEMAIS CASOS, O INTERVALO CONTÉM O ZERO.

B	$K=2$	$K=4$	$K=6$	$K=8$	$K=10$
10^2	18	19†	19	18†	14†
10^3	**26**	**29**	21	**26**	24
10^4	**29**	**27**	**25**	**28**	26
10^5	**27**	**27**	**28**	26	**28**
10^6	**29**	**27**	**29**	**28**	**29**

E. Resultados

A Tabela III resume as 750 comparações.

Com $B = 10^2$, a energia média da busca aleatória é menor em todas as complexidades, e o intervalo de confiança favorece a busca aleatória para $K = 4$, 8 e 10. Ainda assim, o *Simulated Annealing* vence a maioria dos pares em quatro dos cinco casos. Isso indica que ele perde por uma margem grande em poucas execuções. Para $K = 2$, por exemplo, o desvio padrão da sua energia é 0,75, contra 0,24 da busca aleatória. Com apenas 100 avaliações e passos de desvio padrão 0,6, o *Simulated Annealing* explora pouco além da vizinhança da pose inicial, enquanto a busca aleatória sorteia poses em toda a região.

A partir de $B = 10^4$, o *Simulated Annealing* vence entre 25 e 29 dos 30 pares em todas as complexidades, com p-valor do teste dos sinais de no máximo $3{,}3 \times 10^{-4}$. O intervalo de confiança favorece o método em 13 dos 15 casos. Nos outros dois, $K = 10$ com $B = 10^4$ e $K = 8$ com $B = 10^5$, o intervalo contém o zero, mas o número de vitórias continua alto. As diferenças médias de energia são pequenas, abaixo de 0,02, porque os dois métodos chegam perto da mesma região de energia mínima. A vantagem do *Simulated Annealing* está no refinamento local dessa região, que a amostragem uniforme não faz.

O tempo medido também confirma a análise da Seção V-B. Para $K = 2$, o *Simulated Annealing* levou em média 0,61 ms com $B = 10^2$ e 65,3 ms com $B = 10^4$, cerca de 100 vezes mais para um orçamento 100 vezes maior. Com $B = 10^4$ fixo, o tempo passou de 65,3 ms para 130,0 ms quando K foi de 2 para 10, um crescimento aproximadamente linear. O tempo não dobra a cada vez que K dobra porque parte do custo de cada iteração não depende de K, como a rotação dos quatro átomos e o sorteio da vizinha. Com o mesmo orçamento, o *Simulated Annealing* levou entre 1,2 e 1,5 vez o tempo da busca aleatória, o que é coerente com o trabalho extra de cada iteração, que inclui a temperatura e o critério de Metropolis.

F. Limitações do Modelo

O modelo é exclusivamente educacional. O ligante tem apenas quatro átomos no plano, a energia não corresponde a um modelo físico-químico validado, não há flexibilidade interna da molécula e os alvos e obstáculos são sorteados. Por isso, os resultados não se estendem a aplicações farmacêuticas reais. Também é importante separar duas dificuldades. O *docking* real é computacionalmente difícil, pois envolve a flexibilidade do ligante e do receptor e um número muito grande de configurações [2], [3]. Já o modelo simplificado tem avaliação de custo $\Theta(K)$ e não foi provado NP-difícil. O estudo mostra o comportamento de uma metaheurística em um espaço contínuo com mínimos locais, mas não demonstra a intratabilidade do *docking*.

VI. CONCLUSÕES

Este trabalho uniu o estudo teórico dos Capítulos 34 e 35 de Cormen et al. [1] com testes práticos de algoritmos exatos, aproximados e metaheurísticos em Python. A partir dos experimentos executados no repositório e do estudo de caso da Seção V, foi possível observar na prática quando cada abordagem funciona bem e quando deixa de ser viável.

Nos algoritmos exatos e verificadores, ficou evidente a diferença entre verificar e resolver um problema. Enquanto os verificadores validam uma solução de TSP em $O(n)$ e de *Vertex Cover* em $O(|V| + |E|)$ em poucos microssegundos, a força bruta trava logo em entradas pequenas. No TSP com $n = 10$, a Força Bruta levou 1,522 s, enquanto o algoritmo de Held-Karp [7] ($\Theta(n^2 2^n)$) levou 6,92 ms (220 vezes mais rápido) e resolveu $n = 14$ em 235,1 ms. Na Cobertura de Vértices com $n = 18$, o *Branch-and-Bound* caiu de 242,1 ms (força bruta) para 4,97 ms. Mesmo assim, métodos exatos só servem para entradas pequenas: acima de $n = 22$, a tabela $\Theta(n2^n)$ do Held-Karp estoura a memória RAM.

Já os algoritmos de aproximação rodaram em microssegundos e mantiveram o erro dentro dos limites teóricos do Capítulo 35: no `APPROX-VERTEX-COVER` ($\rho = 2$), a razão média ficou entre 1,42 e 1,70; no `APPROX-TSP-TOUR` ($\rho = 2$ com desigualdade triangular), as rotas ficaram em média de 8% a 20% acima do ótimo; e no `GREEDY-SET-COVER` ($\rho = H(d)$), a razão média ficou entre 1,00 e 1,06.

Por fim, no problema contínuo inspirado em *Molecular Docking* [2], [3], o *Simulated Annealing* dependeu diretamente do orçamento de avaliações B. Com poucas avaliações ($B = 100$) e passos locais curtos ($\sigma = 0{,}6$), o algoritmo explora pouco além da vizinhança da pose inicial e perde em média para a busca aleatória, que sorteia pontos por todo o espaço. Já para $B \geq 10.000$, o resfriamento gradual e o refinamento local permitiram sair de mínimos locais e superar a busca aleatória em até 29 das 30 execuções ($p \leq 3{,}3 \times 10^{-4}$). A Tabela IV resume onde cada método se aplica.

Tabela IV
RESUMO DE ONDE CADA ABORDAGEM FUNCIONA E ONDE FALHA.

Abordagem	**Onde Usar**	**Onde Falha**
Exatos (Força Bruta, B&B, Held-Karp)	Entradas pequenas ($n \leq 20$) e testes que exigem o ótimo exato C^*.	Explosão de tempo ($2^n, n!$) e falta de memória RAM ($\Theta(n2^n)$).
Aproximação (Cap. 35)	Grafos grandes quando há propriedade estrutural (ex: desigualdade triangular).	Problemas sem aproximação constante (TSP geral) ou domínios contínuos.
Metaheurísticas (*Simulated Annealing*)	Espaços contínuos ou grandes com vários mínimos locais (*Docking*).	Quando se exige garantia formal de erro ou o orçamento B é muito curto.

REFERÊNCIAS

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Algoritmos: Teoria e Prática*, 3rd ed. Rio de Janeiro, Brasil: Elsevier, 2012.

[2] N. S. Pagadala, K. Syed, and J. Tuszynski, "Software for molecular docking: A review," *Biophys. Rev.*, vol. 9, no. 2, pp. 91–102, 2017.

[3] S.-Y. Huang, "Comprehensive assessment of flexible-ligand docking algorithms: Current effectiveness and challenges," *Briefings Bioinf.*, vol. 19, no. 5, pp. 982–994, 2018.

[4] S. A. Cook, "The complexity of theorem-proving procedures," in *Proc. 3rd Annu. ACM Symp. Theory of Computing (STOC)*, 1971, pp. 151–158.

[5] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*. New York, NY, USA: Plenum Press, 1972, pp. 85–103.

[6] L. A. Levin, "Universal sequential search problems," *Problemy Peredachi Informatsii*, vol. 9, no. 3, pp. 115–116, 1973.

[7] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *J. Soc. Ind. Appl. Math.*, vol. 10, no. 1, pp. 196–210, 1962.

[8] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

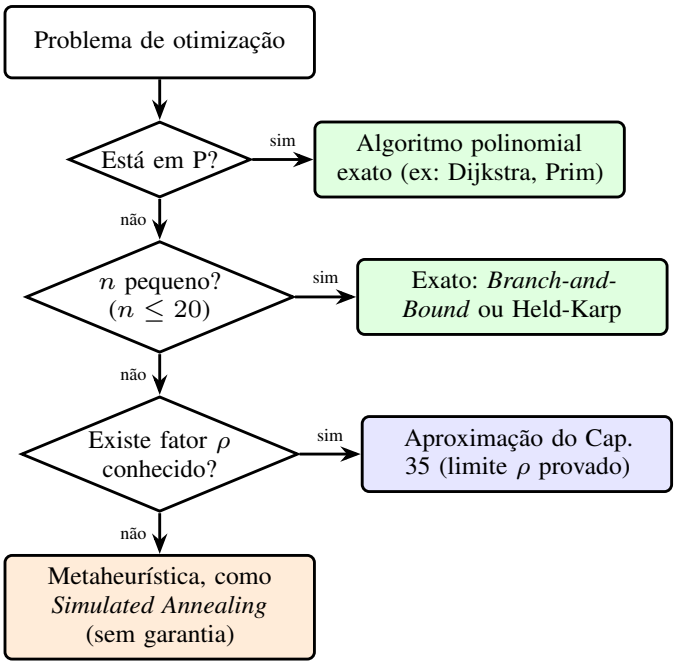
[9] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *J. Chem. Phys.*, vol. 21, no. 6, pp. 1087–1092, 1953.

APÊNDICE A
CHEAT SHEET: GUIA DE CONSULTA RÁPIDA

a) Definição em Uma Frase

Quando um problema tem verificação rápida em tempo polinomial (NP), mas achar o ótimo exato sofre explosão combinatória (NP-Completo), usam-se métodos exatos com poda apenas para entradas pequenas e recorre-se a aproximações (com limite $\rho(n)$) ou metaheurísticas para entradas maiores.

b) Quando Utilizar Cada Abordagem?



c) Tabela Resumo de Complexidades

Algoritmo	Tempo (Pior Caso)	Espaço	Garantia
Verificador Vertex Cover	$O(\|V\| + \|E\|)$	$O(\|V\|)$	Exata (Decisão)
Verificador TSP	$O(n)$	$O(n)$	Exata (Decisão)
Vertex Cover (Força Bruta)	$O(2^n \cdot \|E\|)$	$O(n)$	Ótimo (C^*)
Vertex Cover (Branch-and-Bound)	$O(2^k \cdot \|E\|)$	$O(n)$	Ótimo (C^*)
TSP (Força Bruta)	$\Theta((n-1)! \cdot n)$	$O(n)$	Ótimo (C^*)
TSP (Held-Karp / PD)	$\Theta(n^2 2^n)$	$\Theta(n2^n)$	Ótimo (C^*)
`APPROX-VERTEX-COVER`	$O(\|V\| + \|E\|)$	$O(\|V\|)$	$\rho = 2$
`APPROX-TSP-TOUR`	$\Theta(n^2)$	$O(n)$	$\rho = 2$ (métrico)
`GREEDY-SET-COVER`	$O(\|X\|\|\mathcal{F}\|\min(\|X\|,\|\mathcal{F}\|))$	$O(\|X\|)$	$\rho = H(d)$
Simulated Annealing	$\Theta(B \cdot K)$	$O(K)$	Empírica

No *Simulated Annealing*, B é o orçamento de avaliações e K é o parâmetro de complexidade da instância.

d) Esqueleto Base da Técnica

Todos os algoritmos de aproximação gulosos deste trabalho seguem o mesmo modelo, que aparece no Algoritmo 7, e todas as provas de fator seguem os passos abaixo.

Esqueleto da prova do fator ρ:

1) Encontre um limite inferior L para o ótimo sem calcular o ótimo, como $|A|$ no *Vertex Cover* ou $c(T)$ no TSP.
2) Mostre que $L \leq C^*$.
3) Mostre que a resposta do algoritmo vale $C \leq \rho \cdot L$.
4) Conclua que $C \leq \rho \cdot L \leq \rho \cdot C^*$.

Algoritmo 7: Esqueleto de uma aproximação gulosa

Entrada: Instância I de um problema de minimização.
Saída: Solução viável sol com $C \leq \rho \cdot C^*$.

```
sol ← ∅
enquanto sol ainda não é viável faça
    Escolha o item de maior ganho local
    sol ← sol ∪ {item}
    Atualize o que ainda falta resolver
retorna sol
```